Smartphone Domain Word2Vec Project

Overview

This project demonstrates how to build and evaluate **Word2Vec word embeddings** using a custom **smartphone/mobile domain corpus** containing **100,000 sentences**. The notebook trains and compares two Word2Vec architectures: **CBOW (Continuous Bag of Words)** and **Skip-gram**. After training, the learned embeddings are evaluated using **cosine similarity** to examine semantic relationships between smartphone-related terms such as *camera*, *battery*, *display*, *Samsung*, *Apple*, and *5G*.

The purpose of this project is to understand how word embedding models learn semantic meaning from domain-specific text and how those embeddings can be used to measure similarity between words in a specialized vocabulary. Since the dataset is focused entirely on the smartphone/mobile domain, the trained vectors are expected to capture relationships relevant to devices, features, brands, connectivity, performance, and user experience.

Dataset Description

The dataset used in this project is a **synthetic smartphone/mobile corpus** consisting of **100,000 English sentences**. Each sentence is generated using smartphone-related vocabulary such as:

- **Brands:** Apple, Samsung, Google, OnePlus, Xiaomi, Sony, Nokia, Motorola
- **Device references:** flagship, midrange, budget phone, device, model, series
- **Features:** camera, battery, display, processor, RAM, storage, 5G connectivity, fingerprint sensor, face unlock
- **Usage contexts:** gaming, photography, multitasking, video streaming, daily usage, productivity, social media
- **Descriptive terms:** fast, reliable, efficient, powerful, smooth, responsive, high-quality

Example sentence from the corpus:

> The Samsung flagship improves smooth battery performance for gaming.

The corpus is stored in a plain text file, where each line represents one sentence. This format makes it easy to preprocess and use directly for Word2Vec training.

Project Objectives

The notebook is designed to achieve the following goals:

1. Load and preprocess a smartphone-specific text corpus.
2. Train a **CBOW Word2Vec model** on the corpus.

3. Train a **Skip-gram Word2Vec model** on the same corpus.
4. Compare how both models learn word relationships in a domain-specific setting.
5. Evaluate semantic similarity between smartphone-related terms using **cosine similarity**.
6. Explore nearest-neighbor words for selected terms such as *battery*, *camera*, *Apple*, or *gaming*.

Notebook Workflow

1. Data Loading

The notebook begins by loading the `smartphone_corpus_100k.txt` file. Since the corpus is stored as one sentence per line, each sentence is read and stored as a text instance.

2. Text Preprocessing

The text is cleaned and tokenized before training. Typical preprocessing steps include:

- converting text to lowercase
- removing punctuation if necessary
- splitting each sentence into tokens
- storing the tokenized corpus as a list of lists

For example:

Original sentence
`The Apple flagship improves fast camera performance for photography.`

Tokenized form
`['the', 'apple', 'flagship', 'improves', 'fast', 'camera', 'performance', 'for', 'photography']`

This tokenized representation is then used as input to Word2Vec.

Word2Vec Models

3. CBOW Model

The **Continuous Bag of Words (CBOW)** model predicts a target word from its surrounding context words. It is generally faster to train and works well when the corpus is moderate in size.

For example, in a sentence such as:

> The Samsung flagship improves smooth battery performance for gaming.

CBOW learns to predict the word **battery** from nearby words such as *Samsung*, *flagship*, *improves*, *smooth*, and *performance*.

In the notebook, CBOW is implemented by setting:

```
sg = 0
```

in the Gensim Word2Vec model.

Example configuration:

```
from gensim.models import Word2Vec

cbow_model = Word2Vec(
    sentences=tokenized_sentences,
    vector_size=100,
    window=5,
    min_count=1,
    workers=4,
    sg=0
)
```

Key Parameters

- **vector_size**: dimensionality of word embeddings
- **window**: number of surrounding context words to consider
- **min_count**: minimum frequency required for a word to be included in the vocabulary
- **sg=0**: specifies CBOW training

---

4. Skip-gram Model

The **Skip-gram** model works in the opposite direction: it uses a target word to predict surrounding context words. Skip-gram is often better at learning high-quality embeddings for rare words, though it may take longer to train.

Using the same sentence, Skip-gram would use the word **battery** to predict surrounding words such as *smooth*, *performance*, and *gaming*.

In the notebook, Skip-gram is implemented by setting:

```
sg = 1
```

Example configuration:

```
skipgram_model = Word2Vec(
    sentences=tokenized_sentences,
    vector_size=100,
    window=5,
    min_count=1,
    workers=4,
    sg=1
)
```

Key Parameters

- **vector_size**: dimensionality of embeddings
- **window**: size of context window
- **min_count**: keeps all relevant smartphone terms in vocabulary
- **sg=1**: specifies Skip-gram training

Evaluation Using Cosine Similarity

5. Why Cosine Similarity?

Once word embeddings are trained, each word is represented as a numeric vector in a high-dimensional space. To determine how semantically similar two words are, the notebook uses **cosine similarity**.

Cosine similarity measures the angle between two vectors and returns a score between **-1 and 1**:

- **1** means the vectors are very similar
- **0** means they are unrelated
- **-1** means they are opposite in direction

In practice, words that appear in similar contexts should have higher cosine similarity scores. For example:

- **camera** and **photography** should be similar
- **battery** and **charging** may be similar if both occur in related contexts
- **Apple** and **Samsung** may be close because both are smartphone brands
- **display** and **gaming** may show some relation depending on the training corpus

---

6. Similarity Evaluation in the Notebook

The notebook evaluates the embeddings in two common ways.

A. Pairwise cosine similarity

The similarity between two chosen words can be computed as:

```
similarity = cbow_model.wv.similarity("camera", "photography")
print(similarity)
```

or for Skip-gram:

```
similarity = skipgram_model.wv.similarity("battery", "charging")
print(similarity)
```

This provides a numeric score showing how closely the two words are related in the learned embedding space.

B. Most similar words

The notebook can also retrieve the nearest words to a given term:

```
cbow_model.wv.most_similar("camera", topn=5)
skipgram_model.wv.most_similar("battery", topn=5)
```

This is useful for understanding whether the model learned meaningful domain-specific relationships. For example, the most similar words to **camera** may include *photography*, *display*, *flagship*, or *performance* depending on the corpus patterns.

Expected Results

Since the corpus is focused on smartphone/mobile vocabulary, the learned embeddings should capture relationships such as:

- **brand relationships**: Apple, Samsung, Google, Xiaomi
- **feature relationships**: camera, battery, display, RAM, storage
- **usage relationships**: gaming, photography, productivity, streaming
- **device category relationships**: flagship, midrange, budget phone

The Skip-gram model may produce slightly richer relationships for less frequent or specialized terms, while CBOW may train faster and produce smoother representations overall. Comparing both models helps illustrate the trade-off between training efficiency and representation quality.

Files in This Project

`smartphone_corpus_100k.txt`

The generated smartphone/mobile corpus with 100,000 sentences. Each line contains one sentence.

`word2vec_smartphone_notebook.ipynb`

The main notebook containing:

- dataset loading
- preprocessing and tokenization
- CBOW training
- Skip-gram training
- cosine similarity evaluation
- nearest-neighbor word exploration

Example Notebook Structure

A typical notebook for this project may include the following sections:

1. **Import libraries**
   Load `gensim`, `numpy`, `pandas`, and any text preprocessing libraries.

2. **Load dataset**
   Read the `smartphone_corpus_100k.txt` file.

3. **Preprocess text**
   Lowercase, tokenize, and prepare the sentence list.

4. **Train CBOW model**
   Use Gensim `Word2Vec` with `sg=0`.

5. **Train Skip-gram model**
   Use Gensim `Word2Vec` with `sg=1`.

6. **Evaluate cosine similarity**
   Compare selected smartphone-related word pairs.

7. **Inspect most similar words**
   Print nearest neighbors for target terms such as *camera*, *battery*, and *Apple*.

8. **Compare CBOW vs Skip-gram results**
   Analyze differences in learned relationships and performance.

Example Research Questions

This project can help answer questions such as:

- Do words like **camera** and **photography** appear close in embedding space?
- Are smartphone brands like **Apple** and **Samsung** learned as semantically related?
- Does Skip-gram capture domain-specific feature relationships better than CBOW?
- How well do embeddings reflect the structure of smartphone terminology in a synthetic corpus?

Limitations

This dataset is **synthetic**, which means the sentence patterns are generated from predefined templates rather than collected from real-world mobile reviews, support forums, or product descriptions. As a result:

- vocabulary diversity is limited
- sentence structure is repetitive
- some relationships may reflect template design rather than naturally occurring language

Even with these limitations, the dataset is useful for **learning and demonstrating Word2Vec concepts**, especially for domain-specific embedding experiments.

Possible Extensions

This project can be extended in several ways:

- add **real smartphone reviews** and user comments
- include **technical smartphone terminology** such as AMOLED, chipset, Snapdragon, refresh rate, OTA update, and benchmark
- compare Word2Vec with **FastText**, **GloVe**, or contextual embeddings such as **BERT**
- visualize learned embeddings using **t-SNE** or **PCA**
- evaluate analogies such as:
- Apple : iPhone :: Samsung : Galaxy
- camera : photography :: battery : charging

Conclusion

This project provides a practical introduction to **Word2Vec in a domain-specific setting** using a smartphone/mobile corpus. By training both **CBOW** and **Skip-gram** models, the notebook demonstrates how word embeddings capture semantic relationships between brands, device types, smartphone features, and usage contexts. The use of **cosine similarity** makes it possible to evaluate whether the learned vectors represent meaningful relationships in the mobile domain. Overall, the notebook serves as a useful hands-

on exercise for understanding distributed word representations, training word embedding models, and analyzing semantic similarity in specialized corpora.